

Migración a JWT

Cambios requeridos en el Frontend

LDL-UNSADA-TPI — Integración de Seguridad ICB

Versión: 1.0

Fecha: Junio 2026

Para: Equipo de Frontend

Desde: Equipo de Backend

1. ¿Qué cambia y por qué?

Actualmente la autenticación funciona con **sesiones PHP**: el backend crea una sesión en el servidor y el navegador la mantiene automáticamente mediante una cookie (`PHPSESSID`). Esto tiene limitaciones de escalabilidad y seguridad.

Con **JWT (JSON Web Token)**, el backend ya no guarda estado de sesión. En cada login exitoso se genera un token firmado que el frontend debe almacenar y enviar en cada petición.

Lo que NO cambia:

- Los endpoints de la API siguen siendo los mismos (`?action=login` , `?action=user` , etc.)
- El formato de respuesta sigue siendo `{status, message, data}`
- El control de acceso por rol (admin/user) se mantiene
- El flujo de login, creación de usuarios, mensajería, etc. no cambia

Lo que SÍ cambia:

- Cada `fetch()` debe incluir el header `Authorization: Bearer <token>`
- El login devuelve un `token` adicional que debe guardarse en `sessionStorage`
- Si la API responde con **401**, el token expiró y hay que redirigir al login
- Los endpoints de recuperación de contraseña (`recover_request` , `recover_reset`) son públicos y NO necesitan token

2. Visión general de los cambios

2.1 Archivo nuevo: `frontend/js/api.js`

Se crea un helper centralizado que reemplaza a `fetch()` . Este helper agrega automáticamente el token Bearer a todas las peticiones y redirige al login si recibe un 401.

2.2 Modificaciones en 8 archivos HTML existentes

Cada `fetch()` se reemplaza por una llamada al helper `api()` . En total son **26 llamados** a modificar, todos siguiendo el mismo patrón.

1	8	26	2
Archivo nuevo	HTMLs a modificar	<code>fetch()</code> a reemplazar	HTMLs sin cambios

Archivo	Cambios	Fetch calls
<code>js/api.js</code>	NUEVO	—
<code>login.html</code>	EDITAR	1
<code>credencial.html</code>	EDITAR	1
<code>enviar-consulta.html</code>	EDITAR	1
<code>mensajes-usuarios.html</code>	EDITAR	1
<code>nuevo-usuario.html</code>	EDITAR	4
<code>editar-usuario.html</code>	EDITAR	7
<code>mensajes.html</code>	EDITAR	3
<code>sellos.html</code>	EDITAR	5
<code>admin-usuarios.html</code>	EDITAR	4
<code>reestablecer-contrasena.html</code>	SIN CAMBIOS	0

3. Contrato de API — Lo que el frontend necesita saber

3.1 Login — Ahora devuelve un token

Endpoint: `POST ../index.php?action=login`

Body:

```
{
  "identificador": "usuario_o_dni",
  "password": "contraseña"
}
```

Respuesta exitosa (200):

```
{
  "status": "success",
  "message": "Login exitoso.",
  "data": {
    "id": 1,
    "usuario": "juan",
    "nombre": "Juan",
    "apellido": "Pérez",
    "dni": "12345678",
    "email": "juan@mail.com",
    "rol": "admin",
    "foto_perfil": "images/...",
    "telefono": "11600000000",
    "direccion": "Calle 123",
    "token": "eyJhbGciOiJIUzI1NiIs..."
  }
}
```

NUEVO: El campo `token` contiene el JWT que debe guardarse en `sessionStorage`.

3.2 Demás endpoints — Requieren Authorization header

Todos los endpoints protegidos esperan:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Si el token falta, es inválido o expiró, el backend responde:

```
{
  "status": "error",
  "message": "Token inválido o expirado.",
  "data": []
}
// Código HTTP: 401
```

3.3 Endpoints públicos (NO requieren token)

Endpoint	Método	Uso
?action=login	POST	Iniciar sesión
?action=recover_request	POST	Solicitar código de recuperación
?action=recover_reset	POST	Restablecer contraseña

4. Archivo nuevo: `frontend/js/api.js`

Crear el directorio `frontend/js/` y agregar este archivo. Es el **único cambio repetitivo**: una vez creado, solo hay que reemplazar `fetch()` por `api()` en cada HTML.

`frontend/js/api.js`

```
/**
 * Helper centralizado para llamadas a la API con JWT.
 *
 * Agrega automáticamente el token Bearer a cada petición
 * y redirige al login si el token expiró (HTTP 401).
 *
 * USO:
 *   const result = await api('../index.php?action=user');
 *   const result = await api('../index.php?action=user', {
 *     method: 'POST',
 *     body: JSON.stringify({ ... })
 *   });
 */
async function api(url, options = {}) {
  // Obtener token guardado en el login
  const token = sessionStorage.getItem('jwt_token');

  // Preparar headers: siempre JSON
  options.headers = {
    'Content-Type': 'application/json',
    ...options.headers,
  };

  // Agregar token si existe
  if (token) {
    options.headers['Authorization'] = `Bearer ${token}`;
  }

  // Hacer la petición
  const response = await fetch(url, options);
  const result = await response.json();

  // Si el token expiró (401), redirigir al login
  if (response.status === 401 && !url.includes('action=login')) {
    sessionStorage.clear();
    window.location.href = 'login.html';
    return null;
  }
}
```

```
}  
  
    return result;  
}
```

Notas:

- El helper siempre envía `Content-Type: application/json` — podés sacar los headers manuales en cada HTML
- Si el backend responde 401, limpia `sessionStorage` y redirige automáticamente
- Las respuestas se devuelven como objeto JS directamente (no necesitás `.json()`)

5. Cambios archivo por archivo

En cada HTML hay que:

1. Agregar `<script src="js/api.js"></script>` en el `<head>`
 2. Reemplazar cada `fetch()` por `api()`
 3. En `login.html` además guardar el token
-

5.1 `login.html`

Paso 1: Agregar el script del helper

```
<script src="js/api.js"></script>
```

Paso 2: Reemplazar el fetch de login

ANTES:

```
const response = await fetch('../index.php?action=login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    identificador: identificador,
    password: password
  })
});
const result = await response.json();
```

DESPUÉS:

```
const result = await api('../index.php?action=login', {
  method: 'POST',
  body: JSON.stringify({
    identificador: identificador,
    password: password
  })
});
```


Paso 3: Guardar el token después del login exitoso

ANTES:

```
if (result.status === 'success') {  
    sessionStorage.setItem('usuario', JSON.stringify(result.data));  
  
    if (result.data.rol === 'admin') { ... }
```

DESPUÉS:

```
if (result.status === 'success') {  
    sessionStorage.setItem('usuario', JSON.stringify(result.data));  
    sessionStorage.setItem('jwt_token', result.data.token);  
  
    if (result.data.rol === 'admin') { ... }
```

5.2 credencial.html

Paso 1: Agregar helper

```
<script src="js/api.js"></script>
```

Paso 2: Reemplazar fetch

ANTES:

```
const response = await fetch(  
    `../index.php?action=credencial&id_usuario=${usuario.id}`  
);  
const result = await response.json();
```

DESPUÉS:

```
const result = await api(  
    `../index.php?action=credencial&id_usuario=${usuario.id}`  
);
```

5.3 `enviar-consulta.html`

Paso 1: Agregar helper

```
<script src="js/api.js"></script>
```

Paso 2: Reemplazar fetch

ANTES:

```
const response = await fetch('../index.php?action=message', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    contenido: mensaje,
    id_usuario: usuario.id
  })
});
const result = await response.json();
```

DESPUÉS:

```
const result = await api('../index.php?action=message', {
  method: 'POST',
  body: JSON.stringify({
    contenido: mensaje,
    id_usuario: usuario.id
  })
});
```

5.4 `mensajes-usuarios.html`

Paso 1: Agregar helper

```
<script src="js/api.js"></script>
```

Paso 2: Reemplazar fetch

ANTES:

```
const response = await fetch(  
  `../index.php?action=message&id_usuario=${usuario.id}&rol=user`  
);  
const result = await response.json();
```

DESPUÉS:

```
const result = await api(  
  `../index.php?action=message&id_usuario=${usuario.id}&rol=user`  
);
```

5.5 nuevo-usuario.html — 4 fetch() calls

Paso 1: Agregar helper

```
<script src="js/api.js"></script>
```

5.5a — Cargar sellos disponibles

ANTES:

```
const response = await fetch('../index.php?action=sello');  
const result = await response.json();
```

DESPUÉS:

```
const result = await api('../index.php?action=sello');
```

5.5b — Crear usuario

ANTES:

```
const userResponse = await fetch('../index.php?action=user', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ usuario, password, nombre, apellido, ... })  
});  
const userResult = await userResponse.json();
```

DESPUÉS:

```
const userResult = await api('../index.php?action=user', {  
  method: 'POST',  
  body: JSON.stringify({ usuario, password, nombre, apellido, ... })  
});
```

5.5c — Crear credencial

ANTES:

```
const credResponse = await fetch('../index.php?action=credential', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ id_usuario, fecha_vencimiento, sellos })
});
```

DESPUÉS:

```
await api('../index.php?action=credential', {
  method: 'POST',
  body: JSON.stringify({ id_usuario, fecha_vencimiento, sellos })
});
```

5.5d — Campana de alertas

ANTES:

```
const response = await fetch('../index.php?action=credential&sub=alerts');
const result = await response.json();
```

DESPUÉS:

```
const result = await api('../index.php?action=credential&sub=alerts');
```

5.6 `editar-usuario.html` — 7 fetch() calls

Paso 1: Agregar helper

```
<script src="js/api.js"></script>
```

5.6a — Cargar datos (3 en paralelo)

ANTES:

```
const [sellosResult, userResult, credResult] = await Promise.all([
  fetch('../index.php?action=sello').then(r => r.json()),
  fetch(`../index.php?action=user&id=${userId}`).then(r => r.json()),
  fetch(`../index.php?action=credential&id_usuario=${userId}`).then(r =>
```

```
r.json())
]);
```

DESPUÉS:

```
const [sellosResult, userResult, credResult] = await Promise.all([
  api('../index.php?action=sello'),
  api('../index.php?action=user&id=${userId}`),
  api('../index.php?action=credential&id_usuario=${userId}`)
]);
```

5.6b — Guardar usuario (PUT)

ANTES:

```
const userResponse = await fetch(`../index.php?action=user&id=${userId}`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ nombre, apellido, ... })
});
const userResult = await userResponse.json();
```

DESPUÉS:

```
const userResult = await api(`../index.php?action=user&id=${userId}`, {
  method: 'PUT',
  body: JSON.stringify({ nombre, apellido, ... })
});
```

5.6c — Actualizar credencial (PUT)

ANTES:

```
await fetch(`../index.php?action=credential&id=${credencialActual.id}`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ fecha_vencimiento, sellos })
});
```

DESPUÉS:

```
await api(`../index.php?action=credential&id=${credencialActual.id}`, {
  method: 'PUT',
  body: JSON.stringify({ fecha_vencimiento, sellos })
});
```

5.6d — Crear credencial (POST)

ANTES:

```
await fetch('../index.php?action=credential', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ id_usuario, fecha_vencimiento, sellos })
});
```

DESPUÉS:

```
await api('../index.php?action=credential', {
  method: 'POST',
  body: JSON.stringify({ id_usuario, fecha_vencimiento, sellos })
});
```

5.6e — Campana de alertas

ANTES:

```
const response = await fetch('../index.php?action=credential&sub=alerts');
const result = await response.json();
```

DESPUÉS:

```
const result = await api('../index.php?action=credential&sub=alerts');
```

5.7 mensajes.html — 3 fetch() calls

Paso 1: Agregar helper

```
<script src="js/api.js"></script>
```

5.7a — Cargar mensajes

ANTES: `fetch('../index.php?action=message').then(r => r.json())`

DESPUÉS: `api('../index.php?action=message')`

5.7b — Responder mensaje (PUT)

ANTES:

```
const response = await fetch(`../index.php?action=message&id=${id}`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ respuesta: textarea.value })
});
const result = await response.json();
```

DESPUÉS:

```
const result = await api(`../index.php?action=message&id=${id}`, {
  method: 'PUT',
  body: JSON.stringify({ respuesta: textarea.value })
});
```

5.7c — Campana de alertas

ANTES: `fetch('../index.php?action=credential&sub=alerts').then(r => r.json())`

DESPUÉS: `api('../index.php?action=credential&sub=alerts')`

5.8 sellos.html — 5 fetch() calls

Paso 1: Agregar helper

```
<script src="js/api.js"></script>
```

5.8a — Cargar sellos

ANTES: `fetch('../index.php?action=sello').then(r => r.json())`

DESPUÉS: `api('../index.php?action=sello')`

5.8b — Toggle visible (PUT)

ANTES:

```
const response = await fetch(`../index.php?action=sello&id=${id}`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ visible: sello.visible })
});
const result = await response.json();
```

DESPUÉS:

```
const result = await api(`../index.php?action=sello&id=${id}`, {
  method: 'PUT',
  body: JSON.stringify({ visible: sello.visible })
});
```

5.8c — Eliminar sello (DELETE)

ANTES:

```
const response = await fetch(`../index.php?action=sello&id=${id}`, {
  method: 'DELETE'
});
const result = await response.json();
```

DESPUÉS:

```
const result = await api(`../index.php?action=sello&id=${id}`, {
  method: 'DELETE'
});
```

5.8d — Agregar sello (POST)

ANTES:

```
const response = await fetch('../index.php?action=sello', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ nombre, src: base64, visible: true })
});
const result = await response.json();
```

DESPUÉS:

```
const result = await api('../index.php?action=sello', {
  method: 'POST',
  body: JSON.stringify({ nombre, src: base64, visible: true })
});
```

5.8e — Campana de alertas

ANTES: `fetch('../index.php?action=credential&sub=alerts').then(r => r.json())`

DESPUÉS: `api('../index.php?action=credential&sub=alerts')`

5.9 `admin-usuarios.html` — 4 `fetch()` calls

Paso 1: Agregar helper

```
<script src="js/api.js"></script>
```

5.9a — Cargar datos (2 en paralelo)

ANTES:

```
const [usersResult, credsResult] = await Promise.all([
  fetch('../index.php?action=user').then(r => r.json()),
  fetch('../index.php?action=credential').then(r => r.json())
]);
```

DESPUÉS:

```
const [usersResult, credsResult] = await Promise.all([
  api('../index.php?action=user'),
  api('../index.php?action=credential')
]);
```

5.9b — Eliminar usuario (DELETE)

ANTES:

```
const response = await fetch(`../index.php?action=user&id=${usuarioAEliminar}`
, {
  method: 'DELETE'
});
const result = await response.json();
```

DESPUÉS:

```
const result = await api(`../index.php?action=user&id=${usuarioAEliminar}`, {
  method: 'DELETE'
});
```

5.9c — Campana de alertas

ANTES: `fetch('../index.php?action=credential&sub=alerts').then(r => r.json())`

DESPUÉS: `api('../index.php?action=credential&sub=alerts')`

5.10 `reestablecer-contrasena.html` — SIN CAMBIOS

Este archivo no se modifica. Los endpoints `recover_request` y `recover_reset` son públicos y no requieren token JWT.

6. Checklist de implementación

#	Archivo	Acción	Hecho
1	frontend/js/api.js	Crear el helper	
2	login.html	Agregar script + reemplazar fetch + guardar token	
3	credencial.html	Agregar script + reemplazar fetch	
4	enviar-consulta.html	Agregar script + reemplazar fetch	
5	mensajes-usuarios.html	Agregar script + reemplazar fetch	
6	nuevo-usuario.html	Agregar script + reemplazar 4 fetch()	
7	editar-usuario.html	Agregar script + reemplazar 7 fetch()	
8	mensajes.html	Agregar script + reemplazar 3 fetch()	
9	sellos.html	Agregar script + reemplazar 5 fetch()	
10	admin-usuarios.html	Agregar script + reemplazar 4 fetch()	

Pruebas de verificación

#	Prueba	Resultado esperado	Hecho
1	Login correcto	Redirige según rol, token guardado en sessionStorage	
2	Ver credencial	La credencial se carga con el token	
3	Enviar consulta	El mensaje se envía con el token	
4	CRUD usuario admin	Todas las operaciones funcionan con token	
5	Token expirado	Redirige al login automáticamente	
6	Recuperar contraseña	Funciona sin token (público)	
7	Sin token	API responde 401, frontend redirige a login	

LDL-UNSADA-TPI — Migración JWT — Documento para equipo de Frontend

Versión 1.0 — Junio 2026